

THE COMPLETE HANDS-ON GUIDE

Ollama

Model Customisation with the Modelfile

Stop repeating the same system prompt and settings in every script. In this phase you bake your model's personality, behaviour and parameters into a permanent, shareable model of your own — built from a simple recipe file called a Modelfile.

PHASE 5 OF 6

Model Customisation (Modelfile)

Phase 5 — Model Customisation (Modelfile)

In Phase 3 you wrote a great system prompt — but you had to paste it into every script. What if the model simply *was* that persona, permanently, with your favourite settings baked in? That's what a Modelfile gives you: a short text recipe that turns any base model into a named, reusable, shareable model that behaves exactly how you want, every time.

If you've used Docker, the idea will feel familiar: a Modelfile is to an Ollama model what a `Dockerfile` is to a container image — a declarative blueprint you "build" once and run anywhere.

WHAT YOU'LL LEARN IN THIS PHASE

What a Modelfile is · the `FROM`, `SYSTEM`, `PARAMETER`, `TEMPLATE` and `MESSAGE` instructions · building a model with `ollama create` · importing a GGUF file from Hugging Face · and publishing your model with `ollama push`.

5.1 What is a Modelfile?

5.2 `FROM` — the base model

5.3 `SYSTEM` — bake in a system prompt

5.4 `PARAMETER` — temperature, top_p, top_k, num_ctx

5.5 `TEMPLATE` — the prompt template

5.6 `MESSAGE` — pre-load conversation history

5.7 Creating a model — `ollama create`

5.8 Using a GGUF file from Hugging Face

5.9 Sharing models — `ollama push`

5.10 Hands-on project: build & ship your own model

5.1 — What is a Modelfile?

A **Modelfile** is a plain text file — the blueprint Ollama uses to create and share customised models. It's a list of **instructions**, one per line, each in uppercase by convention followed by its arguments. From this recipe, `ollama create` produces a brand-new model you can `run`, call from Python, list, and push, exactly like any model you pulled.

```
# comment lines start with a hash
INSTRUCTION arguments
```

The full set of instructions, in the order you'll most often use them:

Instruction	Purpose
<code>FROM</code> (required)	The base model to build on.
<code>SYSTEM</code>	A permanent system message (persona / rules).
<code>PARAMETER</code>	A default runtime setting (temperature, context size...).
<code>TEMPLATE</code>	The full prompt template sent to the model.
<code>MESSAGE</code>	Pre-loaded example conversation turns.
<code>ADAPTER</code> / <code>LICENSE</code> / <code>REQUIRES</code>	LoRA adapter, license text, and minimum Ollama version (advanced).

TWO FACTS WORTH KNOWING UP FRONT

The Modelfile is **not case-sensitive** (uppercase is just convention for readability), and **instructions can appear in any order** — though `FROM` is put first by habit. You can always inspect any existing model's recipe with `ollama show --modelfile <model>`, which is the best way to learn by example.

5.2 — `FROM`: the base model

Every Modelfile must start from *something*. The `FROM` instruction names the base you build on. There are three sources.

```
# 1. From an existing Ollama model (most common)
FROM llama3.2

# 2. From a directory of Safetensors weights (a downloaded model)
FROM ./my-model-directory

# 3. From a single GGUF file (e.g. one downloaded from Hugging Face – see 5.8)
FROM ./ollama-model.gguf
```

For GGUF and Safetensors, the path is either absolute or relative to the Modelfile's own location. For an existing model, just use its name and optional tag (`llama3.2:1b`).

5.3 — `SYSTEM` : bake in a system prompt

This is the instruction you'll reach for most. `SYSTEM` permanently embeds the system message into your model, so the persona and rules you crafted in Phase 3 apply automatically — no need to pass them on every call. Wrap multi-line text in triple quotes.

```
FROM llama3.2

SYSTEM """You are Mario from Super Mario Bros, acting as a helpful assistant.
Stay in character, keep answers upbeat, and end with a cheerful 'Let's-a go!'"""
```

WHY THIS IS POWERFUL

Once baked in, anyone (or any script) that runs your model gets the persona for free.

`ollama.chat(model='mario', messages=[...])` needs no system message at all — it's already part of the model.

5.4 — `PARAMETER`: default runtime settings

`PARAMETER` sets a default for how the model runs. These are the same options you could pass per-call via `options={...}` in Python (Phase 2–3) — but baking them in makes them the model's permanent defaults. You can list as many as you like, one per line.

```
FROM llama3.2
PARAMETER temperature 0.7
PARAMETER top_p 0.9
PARAMETER top_k 40
PARAMETER num_ctx 4096
```

The parameters you'll tune most often:

Parameter	What it controls	Default
<code>temperature</code>	Creativity / randomness. Higher = more inventive, lower = more focused and repeatable.	0.8
<code>top_p</code>	Nucleus sampling — considers tokens up to a cumulative probability. Works alongside <code>top_k</code> .	0.9
<code>top_k</code>	Limits choices to the k most likely tokens. Lower = more conservative.	40
<code>num_ctx</code>	Context-window size (how many tokens the model can read at once).	2048
<code>seed</code>	Fix the random seed so the same prompt gives the same output — great for testing.	0
<code>num_predict</code>	Maximum tokens to generate (–1 = unlimited).	–1
<code>stop</code>	Stop sequence(s); the model halts when it produces one. Repeat the line for several.	—
<code>repeat_penalty</code>	How strongly to discourage repetition. Higher penalises repeats more.	1.1

TEMPERATURE IN PLAIN TERMS

Think of `temperature` as a creativity dial. Near `0` the model is precise and predictable (ideal for code, extraction, structured JSON). Near `1` it's imaginative and varied (ideal for brainstorming and creative writing). For a focused assistant, somewhere around `0.6–0.8` is a good starting point.

5.5 — `TEMPLATE`: the prompt template

Behind the scenes, Ollama wraps your messages in a model-specific format using special tokens the model was trained on. The `TEMPLATE` instruction defines that wrapping. It uses **Go template syntax** and three main variables.

Variable	Stands for
<code>{{ .System }}</code>	The system message.
<code>{{ .Prompt }}</code>	The user's message.
<code>{{ .Response }}</code>	The model's reply (text after it is omitted when generating).

```
TEMPLATE """{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
"""
```

USUALLY, LEAVE THIS ALONE

The template is model-specific and easy to get subtly wrong — a broken template produces garbled output. When you build `FROM` an existing Ollama model, it already has the correct template, so you almost never need to write your own. You mainly need `TEMPLATE` when importing a raw GGUF that ships without one (section 5.8). When in doubt, copy the template from `ollama show --modelfile <model>`.

5.6 — `MESSAGE`: pre-load conversation history

Remember few-shot prompting from Phase 3? `MESSAGE` lets you bake those example turns directly into the model, so the behaviour is built in rather than supplied each call. Each line is `MESSAGE <role> <text>`, using the familiar roles.

```
FROM llama3.2
SYSTEM """Answer only 'yes' or 'no'."""

MESSAGE user Is Toronto in Canada?
MESSAGE assistant yes
MESSAGE user Is Sacramento in Canada?
MESSAGE assistant no
MESSAGE user Is Ontario in Canada?
MESSAGE assistant yes
```

The model now has a built-in pattern: terse yes/no answers. Anyone running it inherits that behaviour without seeing the examples — they're part of the model itself.

5.7 — Creating a model: `ollama create`

With your Modelfile written, one command turns the recipe into a real, named model. The `-f` flag points to the Modelfile.

```
# Build a model named "mybot" from ./Modelfile
ollama create mybot -f ./Modelfile

# Now it behaves like any other model
ollama run mybot
ollama list           # mybot now appears here
```

And of course it works from Python immediately, tying back to Phase 2–3:

```
import ollama
r = ollama.chat(model='mybot',
                messages=[{'role': 'user', 'content': 'Hello!'}])
print(r.message.content) # persona + parameters already baked in
```

ITERATING QUICKLY

Editing a Modelfile and re-running `ollama create` with the same name simply rebuilds the model — Ollama reuses the shared base layers, so rebuilds are fast. Tweak, rebuild, test, repeat. You can also create models programmatically with `ollama.create(...)` in Python.

5.8 — Using a GGUF file from Hugging Face

The wider open-model world distributes models as **GGUF** files — a single-file format Ollama can import directly. This unlocks thousands of community models on Hugging Face that aren't in Ollama's own library.

1. Download a `.gguf` file from a Hugging Face model repository (look for files ending in `.gguf`, often with a quantization tag like `Q4_K_M`).
2. Put it beside a Modelfile that points at it with `FROM`.
3. Build it with `ollama create`.

```
# Modelfile
FROM ./mistral-7b-instruct.Q4_K_M.gguf
SYSTEM """You are a concise, helpful assistant."""
PARAMETER temperature 0.7
```



```
ollama create my-mistral -f ./Modelfile
ollama run my-mistral
```

A NOTE ON TEMPLATES FOR RAW GGUF

Some GGUF files don't include a chat template. If your imported model's replies look malformed, add a `TEMPLATE` (section 5.5) matching the model's expected format — the model's Hugging Face page usually documents which prompt format it was trained on (e.g. ChatML, Llama, Mistral).

5.9 — Sharing models: `ollama push`

Once you've built something useful, you can publish it to ollama.com so others (or your other machines) can pull it. The model name must be **namespaced with your username**: `your-username/model-name`.

```
# 1. Sign in once (registers your machine's public key with ollama.com)
ollama signin

# 2. Name (or copy) your model under your username namespace
ollama create yourname/studybuddy -f ./Modelfile

# 3. Push it to the registry
ollama push yourname/studybuddy
```

Anyone can then install it with the usual `ollama pull yourname/studybuddy`. You can also push from Python with `ollama.push('yourname/studybuddy')`.

PREREQUISITES FOR PUSHING

You need a free account on ollama.com and your machine's public key registered (handled by `ollama signin`, or via the Keys page in your account settings — see Phase 1's FAQ notes on the public key location). The model name's prefix must exactly match your username.

Build & Ship "StudyBuddy" — your own named model

Unlike previous phases, this project's main artefact is a **Modelfile**, not a Python script. You'll combine `FROM`, `SYSTEM`, `PARAMETER` and `MESSAGE` into one coherent custom model — a friendly study tutor with a fixed persona, sensible settings, and a baked-in answer style — then build it, test it from both the CLI and Python, and optionally publish it.

Step 1 — Write the Modelfile

Create a file named `Modelfile` (no extension) with this content:

```
# Modelfile – StudyBuddy, a friendly study tutor
FROM llama3.2

# --- persona & rules (5.3) ---
SYSTEM """You are StudyBuddy, an encouraging tutor for students.
Rules:
- Explain ideas simply, using one short everyday analogy.
- Keep answers under 120 words.
- Always finish with one quick check-question to test understanding.
Never just give the answer to homework; guide the student to it."""

# --- baked-in defaults (5.4) ---
PARAMETER temperature 0.6
PARAMETER num_ctx 4096

# --- few-shot examples that set the answer style (5.6) ---
MESSAGE user What is a variable in programming?
MESSAGE assistant Think of a variable like a labelled box: you put a value
inside and the label lets you find it later. For example, score = 10 stores 10
under the name "score". Quick check: if you wrote name = "Sam", what's inside
the box labelled name?
```

Step 2 — Build it

```
ollama create studybuddy -f ./Modelfile
ollama list          # confirm "studybuddy" appears
```

Step 3 — Test from the CLI

```
ollama run studybuddy "Explain what an array is."
```

Notice you never set a system prompt — the persona, the 120-word limit, and the check-question style all come from the model itself.

Step 4 — Test from Python

```
# test_studybuddy.py
import ollama

r = ollama.chat(
    model='studybuddy',
    messages=[{'role': 'user', 'content': 'What is a loop?'}],
)
print(r.message.content) # tutor persona, baked in – no system prompt needed
```

Step 5 — (Optional) Publish it

```
ollama signin
ollama create yourname/studybuddy -f ./Modelfile
ollama push yourname/studybuddy
```

EXTEND IT YOURSELF

(1) Add a `stop` parameter so it never rambles past its check-question. (2) Make a second variant `FROM studybuddy` with a higher temperature for "creative writing coach" mode. (3) Import a small GGUF from Hugging Face and give *it* the StudyBuddy persona.

What you achieved

You turned a base model into your own named model with a permanent persona, default parameters and few-shot behaviour — built it with `ollama create`, ran it from the CLI and Python with zero per-call setup, and learned how to import GGUF models and publish your own. Your prompts from Phase 3 are now reusable, shareable products.

COMING NEXT — PHASE 6: ADVANCED TOPICS

The finale: embeddings and cosine similarity, calling the REST API directly with `requests` and `httpx`, streaming newline-delimited JSON, building a full RAG pipeline, multi-modal models with images, and running Ollama as a network service.